



UNIVERSITY OF COLOGNE

THE BITSYNC ALGORITHM

Martin Werner

2026

1 Introduction

The name of the algorithm 'BitSync' might be a bit confusing, however, it is derived from a procedure that we call 'bitwise synchronization'. It indeed checks all bits that, as a representation for vertices $v \in M \times W$,¹ are shared by other vertices in order to create fully stable chains c and, as a result, outputs all stable matches at given n .

2 Preliminaries

2.1 Definitions

Definition 1 (Match). *An individual match μ is an assignment of an agent $a \in A$ to another agent $b \in B$ to form a pair $\{a, b\}$ in a two-sided matching model where $A \cap B = \emptyset$.² The general notation for such a match between a and b is $a = \mu(b)$ as well as $b = \mu(a)$.³ A final matching μ over the sets A and B is a set of pairs $\{a, b\}$ where $|\mu| = \min\{|A|, |B|\}$.⁴*

Definition 2 (Preference relation set). *A preference relation set $\succ$ is an ordered set of agents $a \in A$ in the preference list of agent $b \in B$, $A \cap B = \emptyset$, where the following holds: If b prefers a over a' , then $a \succ_b a'$, $\forall a, a' \in A, \forall b \in B$. By transitivity in ordered sets, if $a \succ_b a'$ and $a' \succ_b a''$, then $a \succ_b a''$, $\forall a, a', a'' \in A, \forall b \in B$, such that a preference relation set can be created for b : $a_1 \succ_b a_2 \succ_b \dots \succ_b a_{|A|}$.⁵*

Definition 3 (Stable Marriage Problem). *The stable marriage problem $SMP(M, W, \succ)$ with the input variables $M = \{m : \text{men}\}$, $W = \{w : \text{women}\}$ and $\succ = \{\succ_M, \succ_W\}$ with $\succ_M = \{\succ_m : m \in M\}$, $\succ_m = \{w : \text{ordered set of } w \in W\}$ as well as $\succ_W = \{\succ_w : w \in W\}$, $\succ_w = \{m : \text{ordered set of } m \in M\}$ is defined as a bipartite assignment problem where each man m is matched with woman w such that there is no blocking pair $\{m', w'\}$ satisfying $m' \succ_w \mu(w)$ and $w' \succ_m \mu(m)$, $\forall m, m' \in M, \forall w, w' \in W, w \neq w', m \neq m'$ and $M \cap W = \emptyset$.*

Definition 4 (Vertex). *Let $m \in M, w \in W$ and $SMP(M, W, \succ)$ be a stable marriage problem. The pair $\{m, w\}$ is called a vertex $v \in V, V \subseteq M \times W$ in a non-bipartite⁶ matching graph $G(V, E)$ of $SMP(M, W, \succ)$.*

¹We define, on the contrary to most contributions in the literature, the set of vertices $v \in V$ over the Cartesian product $M \times W$ due to the specific handling of BitSync algorithm. So we have $|V| = |M \times W| = |M| \cdot |W|$ instead of $|V| = |M| + |W|$ as it is usual in bipartite graphs $G(V, E)$.

² A and B are the generalization of sets of agents for topic-related matching models like labor markets with firms and workers, school admissions with schools and students, marriage markets with men and women or organ donor schemes with organ donors and recipients. In the following context, we refer to marriage markets with sets of men M and women W .

³In the literature, μ is generally used for a match of a set of agents to another set of agents where each agent of one set is distinctly matched to another agent of the other set.

⁴If $|B| > |A|$, then all $b \in B \setminus A$ remain unmatched in one-to-one matching.

⁵Note that, here, a_k denotes a 's k -th order of appearance in b 's preference list.

⁶Note that we have vertices over pairs, not single agents. So our graph is defined as a grid over $M \times W$ with vertices $\{m, w\}$ connected to each other.

Definition 5 (Edge). Let $u, v \in V$ with $u = \{m, w\}$ and $v = \{m', w'\}$, $m, m' \in M$, $m \neq m'$, $w, w' \in W$, $w \neq w'$, in a matching graph $G(V, E)$ over $SMP(M, W, \succ)$. The directed edge $e_{u,v} \in E$ is a feasible arrow from vertex u to v .

Definition 6 (Chain). Let $G(V, E)$ be the matching graph in $SMP(M, W, \succ)$ with $v \in V$. The subset of vertices $c = \{v : v \in V\}$ is called chain, the chain length is denoted by $|c|$. The set of all possible chains in $SMP(M, W, \succ)$ is denoted by C .

The chain c is stable if all vertices have stable linkages to each other such that for any two vertices $u, v \in c$ there are no blocking pairs. If we refer to a $|c|$ -chain, then we speak of a chain with a length of $|c|$.

Definition 7 (Rank). The rank $r_m(w)$ is defined as the position of w 's appearance in m 's preference list. The rank $r_w(m)$ is defined as the position of m 's appearance in w 's preference list, respectively. The formal consequence is $r_m(w) < r_m(w')$ iff $w \succ_m w'$ as well as $r_w(m) < r_w(m')$ iff $m \succ_w m'$.

Definition 8 (Rank matrix). The rank matrix $\xi \in \mathbb{R}^2$ is defined as $\xi = \{(r_{m_i}(w_j), r_{w_j}(m_i)) : m_i \in M \text{ in row } i \text{ and } w_j \in W \text{ in column } j, i, j \in [1, n], n = |M| = |W|\}$.

Definition 9 (Stability). In two-sided matching, a chain of two pairs $c = \{\{m, w\}, \{m', w'\}\}$ is said to be stable if and only if there is no blocking pair such that one of the following two conditions hold:

- $w' \succ_m w$ and $m \succ_{w'} m'$
- $w \succ_{m'} w'$ and $m' \succ_w m$

Definition 10 (Stability function). Let $f : C \mapsto \{0, 1\}$ be the stability function that maps a chain $c \in C$ to a Boolean value for indicating whether c is stable or unstable.

That is,

$$f(c) = \begin{cases} 1 & \text{if } c \text{ is stable} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

Definition 11 (Stability matrix). The stability matrix $\Gamma = \{f(u_y, v_x) : u_y, v_x \in V, x, y \in [1, |V|]\}$ is a 2-dimensional matrix over the Cartesian product $V \times V$ that indicates the stability of a 2-length chain $c = \{u_y, v_x\}$ of u_y in row y and v_x in column x . Hence, it is a mapping $\Gamma : V \times V \mapsto \{0, 1\}$.⁷

2.2 Stability computation

Lemma 1 (Stability of all vertices in chain c). Let $u, v, w \in M \times W$. The chain $c = \{u, v, w\}$ is stable, i.e. $f(u, v, w) = 1$, if and only if $f(u, v) = 1$, $f(v, w) = 1$ and $f(u, w) = 1$.

⁷Note that by definition (4), $V \times V = (M \times W) \times (M \times W)$.

For examining whether a chain c is stable or not stable, we have to conclude from definitions (7), (9), (10) that

$$f(c) = \begin{cases} 1 & \text{if } \neg((r_m(w') < r_m(w)) \wedge (r_{w'}(m) < r_{w'}(m'))) \wedge \\ & \neg((r_w(m') < r_w(m)) \wedge (r_{m'}(w) < r_{m'}(w'))) \\ & \forall \{m, w\}, \{m', w'\} \in c, m \neq m', w \neq w' \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

Definition 12. (*Default values for self-reference*) Let $u = \{m, w\}$, $v = \{m, w'\}$ and $w = \{m', w\}$, $u, v, w \in M \times W$, $m, m' \in M$, $w, w' \in W$, $m \neq m'$ and $w \neq w'$. By default, we set $f(u, v) = 0$ and $f(u, w) = 0$ in one-to-one matching since any $m \in M$ or any $w \in W$ can take part only once in a potential match $\mu \in \mathcal{M}$.

For many-to-one matching, we set $f(u, v) = 1$ and $f(u, w) = 0$ by default under the assumption that $m \in M$ has a quota $q_m \geq 1$ and $q_w = q_{w'} = 1$ for $w, w' \in W$.

For many-to-many matching, we set $f(u, v) = 1$ and $f(u, w) = 1$ by default under the assumption that both $m \in M$ and $w \in W$ have a quota $q_m \geq 1$ and $q_w \geq 1$.

2.3 Bit vectors

Before we explain the algorithm's procedure, we want to outline some basics about bit vectors and their numerical representations. A bit vector b with length $|b|$ consists of zeros (0) and ones (1) as core values, its numerical representation $\lambda(b)$ is determined by the following equation:

$$\lambda(b) = \sum_{k=1}^{k=|b|} b_{|b|-k+1} \cdot 2^{k-1} \quad (3)$$

Let $b_k \in \{0, 1\}$, $\forall k \in [1, |b|]$, be a single bit in b . Here, b_1 determines the first element (bit) in b on the left, $b_{|b|}$ the last element (bit) in b on the right.

E.g. if our bit vector $b = \{0, 1, 1, 1, 0, 1\}$, then we have $\lambda(b) = 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 0 + 16 + 8 + 4 + 0 + 1 = 29$.

2.4 Intersection of bit vectors

It is commonly known that bit operations are the foundation of modern devices, mechanisms, electro-technical architectures, algorithms or, let us simply say, everything that can be found in the world of today's technology.

We want to briefly introduce the bitwise intersection operation that exists besides the bitwise union, exclusive intersection, exclusive union, negation, addition, subtraction, multiplication, division and some other specialized operations.

Let $b, b' \in \{0, 1\}$ be two bits that are intersected. The result of the intersection $b \wedge b' = 1$ iff $b = 1$ and $b' = 1$. Otherwise $b \wedge b' = 0$ (see table 1).

$\wedge$	0	1
0	0	0
1	0	1

Table 1: Bitwise intersection

b	$[1011011]$
b'	$[1101101]$
$b \wedge b'$	$[1001001]$

Table 2: Intersection of two bit vectors b and b'

For the intersection of two bit vectors $b, b' \in \{0, 1\}^{|b|}$, $|b| = |b'|$, every two bits $b_k \in b, b'_k \in b'$, are intersected at index $k \in [1, |b|]$ such that $b \wedge b' = \{b_k \wedge b'_k : b_k \in b, b'_k \in b', \forall k \in [1, |b|] \text{ within the ordered sets of } b \text{ and } b'\}$.

E.g. if we have $b = [1011011]$ and $b' = [1101101]$, $|b| = |b'| = 7$, then the intersection of both bit vectors is $b \wedge b' = [1001001]$ as shown in table 2.

Their numerical representations $\lambda(b) = 91$ and $\lambda(b') = 109$ can be intersected as well: $\lambda(b) \wedge \lambda(b') = 91 \wedge 109 = 73$.

Lemma 2. (*Maximum intersection value*) Let $\lambda(b)$ and $\lambda(b')$ be the numerical representations of two bit vectors $b, b' \in \{0, 1\}^{|b|}$, $|b| = |b'|$. It holds that $\lambda(b \wedge b') \leq \min\{\lambda(b), \lambda(b')\}$.

Proof. Since the intersection of the binary values of b and b' leads to the same or a higher amounts of bits that are valued 0, its numerical representation will decrease as a consequence. If $\lambda(b) > \lambda(b')$, then either the first bit of b valued 1 occurs earlier than the first 1-valued bit of b' or b has at least one bit valued 1 occurring at the earliest where b' has a 0-valued bit at the same position after a sequence of same-valued bits for both b and b' . The bitwise intersection leads to 0-valued bits if the values of both bits of b and b' are different at the same position. As a consequence, the intersection value $\lambda(b \wedge b')$ depends on $\lambda(b')$ if $\lambda(b) \geq \lambda(b')$ and, thus, the value of $\lambda(b \wedge b')$ can maximally reach the value of $\lambda(b')$. If we have multiple bitwise intersection operations over random bit vectors, then the result of these operations converges to 0 s.t. $\lim_{i \rightarrow \infty} (\bigwedge_i \lambda(b)) = 0$.⁸

3 Stability matrix and its numerical values

Our stability matrix Γ comprises all necessary bit vectors to determine all stable pairs in $(M \times W) \times (M \times W)$. The Cartesian product $M \times W$ allows us to assign a numerical value to $\{m, w\} \in M \times W$ and vice versa, i.e. from a numerical value $y \in [0, |M \times W| - 1]$, we can identify the according pair $\{m_i, w_j\}, i \in [1, |M|], j \in [1, |W|]$ by the following equation:

⁸The iterator for bitwise intersection operations is denoted by $\bigwedge$.

$$i = \left\lfloor \frac{y}{n} \right\rfloor + 1 \quad (4)$$

$$j = (y \bmod n) + 1 \quad (5)$$

We can divide one bit vector b into n parts⁹ and translate our divided bit vector into numerical values as a representation for the stability of a vertex $v \in M \times W$ w.r.t. another vertex belonging to $m', m' \notin v$.

We define $\lambda(\Gamma)$ as the numerical representation of our binary stability matrix Γ . For each row y in $\lambda(\Gamma)$, we have exactly n columns determining the stability of one vertex $v \in V, m \in v$ to another vertex $v' \in V, m' \in v'$ and $m \neq m'$. We further define $\lambda_x(\Gamma_y)$ the x -th column of the numerical vector of row y of our stability matrix $\lambda(\Gamma)$ and $b_x(\Gamma_y)$ the x -th bit subvector of the binary row Γ_y .¹⁰

Let $\dim_y(A)$ be the total number of rows of matrix A and let $\dim_x(A)$ be the total number of columns of matrix A . By this notation and the general assumption $n = |M| = |W|$, i.e. the sets of M and W are of equal size, $\dim_y(\Gamma) = n^2$, $\dim_x(\Gamma) = n^2$ and for the numerical representation, $\dim_y(\lambda(\Gamma)) = n^2$ and $\dim_x(\lambda(\Gamma)) = n$.

Let s be the bit size for dividing b into smaller parts such that every subvector b_x of b , $x \in \mathbb{N}$, is of size $|b_x| = s$.¹¹ In general, it is possible to use a different bit size s for dividing a bit vector b into several parts.¹² Here, we have chosen $s = n$ for simplicity. However, our test results show that by setting a higher s , we can obtain a higher speed of operation until a certain degree.¹³

Lemma 3. (*Vertex removal*) Suppose that every $m \in M$ and every $w \in W$ is matched under $SMP(M, W, \succ)$. Let $s = n = |M| = |W|$. Let $\lambda_x(\Gamma_y)$ be the x -th column of the numerical vector of row y of the stability matrix $\lambda(\Gamma)$. If $\exists x \in [1, n]$ and if $\exists y \in [0, \dim_y(\Gamma) - 1]$ for $x \neq \lfloor y/n \rfloor + 1$ such that $\lambda_x(\Gamma_y) = 0$, then $v \in M \times W$ represented by $\lambda_x(\Gamma_y)$, that is $v = \{m_i, w_j\}$,¹⁴ is not part of any stable match $\mu \in \mathcal{M}$ under $SMP(M, W, \succ)$.

Proof. If $s = n$, then $\lambda_x(\Gamma_y)$ indicates the stability of $v = \{m_{\lfloor y/n \rfloor + 1}, w_{(y \bmod n) + 1}\}$ to another $v' \in M \times W, m_x \in v'$. If $\lambda_x(\Gamma_y) = 0, x \neq \lfloor y/n \rfloor + 1$, then the according partial bit vector $b_x(\Gamma_y) = \{0\}^s$, i.e. $b_x(\Gamma_y)$ only consists of bits valued by 0, i.e. there are no stable linkages to $v, \forall v' \in M \times W, m_x \in v'$. The default values (see definition 12) are exempted from this consideration.

⁹Suppose $n = |M| = |W|$.

¹⁰Please distinguish the subvector $b_x(\Gamma_y)$ from the single bit b_k in the introductory explanations of bit operations, see equation 3.

¹¹If $|b| \bmod s > 0$, then the last subvector $b_{\lceil |b|/s \rceil}$ of b is of size $|b| \bmod s$.

¹²If $|b| \bmod s = 0$, then there are $|b|/s$ subvectors of b , otherwise we have $\lfloor |b|/s \rfloor + 1$ subvectors of b .

¹³If too much bits are processed at once, our processing behavior will decrease in its speed and, thus, it is not recommended to have high s . So we need an appropriate balance between s and $|b|/s$.

¹⁴See the computations in 4.

$m_1: w_2 \succ w_1 \succ w_5 \succ w_3 \succ w_4$	$w_1: m_3 \succ m_1 \succ m_5 \succ m_4 \succ m_2$
$m_2: w_5 \succ w_1 \succ w_4 \succ w_2 \succ w_3$	$w_2: m_5 \succ m_3 \succ m_1 \succ m_4 \succ m_2$
$m_3: w_5 \succ w_2 \succ w_1 \succ w_4 \succ w_3$	$w_3: m_5 \succ m_2 \succ m_3 \succ m_1 \succ m_4$
$m_4: w_3 \succ w_4 \succ w_2 \succ w_1 \succ w_5$	$w_4: m_1 \succ m_4 \succ m_3 \succ m_2 \succ m_5$
$m_5: w_4 \succ w_1 \succ w_5 \succ w_2 \succ w_3$	$w_5: m_1 \succ m_3 \succ m_2 \succ m_5 \succ m_4$

Table 3: Preferences for $n = 5$

	w_1	w_2	w_3	w_4	w_5
m_1	(2, 2)	(1, 3)	(4, 4)	(5, 1)	(3, 1)
m_2	(2, 5)	(4, 5)	(5, 2)	(3, 4)	(1, 3)
m_3	(3, 1)	(2, 2)	(5, 3)	(4, 3)	(1, 2)
m_4	(4, 4)	(3, 4)	(1, 5)	(2, 2)	(5, 5)
m_5	(2, 3)	(4, 1)	(5, 1)	(1, 5)	(3, 4)

Table 4: Rank matrix ξ at $n = 5$

So if there is $\lambda_x(\Gamma_y) = 0$ for some $x \in [1, n], y \in [0, |M \times W| - 1]$, then we can remove row y from our processing table or interrupt the processing when we reach vertex $v \in V$ at row y .

Example 1. Consider the preference lists shown in table 3. The rank matrix ξ (see table 4) can be constructed from the given preferences $\succ$ and, from the rank matrix ξ , the stability matrix Γ can be computed as a succeeding step (see table 5). Its numerical representation $\lambda(\Gamma)$ is shown in table 7. Note that, by definition 12, we have $\lambda_x(\Gamma_y) = 0$ for $x = \lfloor y/n \rfloor + 1$.¹⁵

By lemma 3, we can detect 0 values for $\lambda_4(\Gamma_3)$, $\lambda_1(\Gamma_6)$, $\lambda_3(\Gamma_9)$, $\lambda_2(\Gamma_{12})$, $\lambda_5(\Gamma_{15})$, $\lambda_1(\Gamma_{16})$, $\lambda_{2,3}(\Gamma_{19})$ and $\lambda_{2,3}(\Gamma_{24})$. So the following rows y can be skipped during our operation: 3, 6, 9, 12, 15, 16, 19 and 24.

The BitSync algorithm produces seven stable matches, see table 6.

4 The algorithm's mode of operation

For this kind of algorithm, we need the upper half Γ^u of the stability matrix Γ . Due to the intersection of bit rows and the fact that we need the whole bit row, displayed as numerical

¹⁵See the framed boxes in the stability matrix $\lambda(\Gamma)$ shown in table 7.

		m_1					m_2					m_3					m_4					m_5				
y	$v \in M \times W$	(m_1, w_1)	(m_1, w_2)	(m_1, w_3)	(m_1, w_4)	(m_1, w_5)	(m_2, w_1)	(m_2, w_2)	(m_2, w_3)	(m_2, w_4)	(m_2, w_5)	(m_3, w_1)	(m_3, w_2)	(m_3, w_3)	(m_3, w_4)	(m_3, w_5)	(m_4, w_1)	(m_4, w_2)	(m_4, w_3)	(m_4, w_4)	(m_4, w_5)	(m_5, w_1)	(m_5, w_2)	(m_5, w_3)	(m_5, w_4)	(m_5, w_5)
0	(m_1, w_1)	0	0	0	0	0	0	0	1	1	1	0	1	0	0	1	0	0	1	1	1	0	1	1	1	1
1	(m_1, w_2)	0	0	0	0	0	1	0	1	1	1	0	0	0	0	1	1	0	1	1	1	1	0	0	1	1
2	(m_1, w_3)	0	0	0	0	0	0	0	0	1	0	1	1	0	1	0	0	0	0	1	0	0	1	0	1	0
3	(m_1, w_4)	0	0	0	0	0	0	0	1	0	0	1	1	1	0	0	0	0	0	0	0	0	1	1	0	0
4	(m_1, w_5)	0	0	0	0	0	0	0	1	1	0	1	1	1	1	0	0	0	1	1	0	0	1	1	1	0
5	(m_2, w_1)	0	1	0	0	0	0	0	0	0	0	0	1	0	0	1	0	1	1	1	0	0	0	0	1	0
6	(m_2, w_2)	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	1	0	0	0	0
7	(m_2, w_3)	1	1	0	1	1	0	0	0	0	0	1	1	0	1	1	1	1	0	1	0	1	1	0	0	0
8	(m_2, w_4)	1	1	1	0	1	0	0	0	0	0	1	1	0	0	1	0	0	1	0	0	1	1	1	0	0
9	(m_2, w_5)	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	0	1	1	1	1	0
10	(m_3, w_1)	0	0	1	1	1	0	0	1	1	0	0	0	0	0	0	0	0	1	1	0	0	1	1	1	0
11	(m_3, w_2)	1	0	1	1	1	1	0	1	1	0	0	0	0	0	0	1	0	1	1	0	1	0	0	1	0
12	(m_3, w_3)	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0
13	(m_3, w_4)	0	0	1	0	1	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	1	1	0	0
14	(m_3, w_5)	1	1	0	0	0	1	1	1	1	0	0	0	0	0	0	1	1	1	1	0	1	1	1	1	0
15	(m_4, w_1)	0	1	0	0	0	0	0	1	0	1	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0
16	(m_4, w_2)	0	0	0	0	0	1	0	1	0	1	0	0	0	0	1	0	0	0	0	0	1	0	0	0	1
17	(m_4, w_3)	1	1	0	0	1	1	1	0	1	1	1	1	0	1	1	0	0	0	0	0	1	1	0	1	1
18	(m_4, w_4)	1	1	1	0	1	1	1	1	0	1	1	1	1	0	1	0	0	0	0	0	1	1	1	0	1
19	(m_4, w_5)	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
20	(m_5, w_1)	0	1	0	0	0	0	1	1	1	1	0	1	0	0	1	0	1	1	1	1	0	0	0	0	0
21	(m_5, w_2)	1	0	1	1	1	0	0	1	1	1	1	0	1	1	1	0	0	1	1	0	0	0	0	0	0
22	(m_5, w_3)	1	0	0	1	1	0	0	0	1	1	1	0	0	1	1	0	0	0	1	0	0	0	0	0	0
23	(m_5, w_4)	1	1	1	0	1	1	0	0	0	1	1	1	0	0	1	0	0	1	0	0	0	0	0	0	0
24	(m_5, w_5)	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0

Table 5: Stability matrix Γ for $n=5$

	m_1	m_2	m_3	m_4	m_5	
μ_1	w_2	w_1	w_5	w_3	w_4	men-optimal
μ_2	w_2	w_4	w_5	w_3	w_1	
μ_3	w_2	w_3	w_5	w_4	w_1	
μ_4	w_1	w_4	w_5	w_3	w_2	
μ_5	w_1	w_3	w_5	w_4	w_2	
μ_6	w_5	w_4	w_1	w_3	w_2	women-optimal
μ_7	w_5	w_3	w_1	w_4	w_2	

Table 6: Seven stable matchings for $n=5$

	m_1	m_2	m_3	m_4	m_5
$x \backslash y$	1	2	3	4	5
0	0	7	9	7	15
1	0	23	1	23	19
2	0	2	26	2	10
3	0	4	28	0	12
4	0	6	30	6	14
5	8	0	9	14	2
6	0	0	1	6	16
7	27	0	27	26	24
8	29	0	25	4	28
9	24	0	0	30	30
10	7	6	0	6	14
11	23	22	0	22	18
12	3	0	0	2	8
13	5	4	0	4	12
14	24	30	0	30	30
15	8	5	9	0	0
16	0	21	1	0	17
17	25	27	27	0	27
18	29	29	29	0	29
19	24	0	0	0	16
20	8	15	9	15	0
21	23	7	23	6	0
22	19	3	19	2	0
23	29	17	25	4	0
24	24	0	0	14	0

Table 7: $\lambda(\Gamma)$ for $n = 5$

	m_1	m_2	m_3	m_4	m_5
$y \backslash x$	1	2	3	4	5
0	0	7	9	7	15
1	0	23	1	23	19
2	0	2	26	2	10
3	0	4	28	0	12
4	0	6	30	6	14
5	0	0	9	14	2
6	0	0	1	6	16
7	0	0	27	26	24
8	0	0	25	4	28
9	0	0	0	30	30
10	0	0	0	6	14
11	0	0	0	22	18
12	0	0	0	2	8
13	0	0	0	4	12
14	0	0	0	30	30
15	0	0	0	0	0
16	0	0	0	0	17
17	0	0	0	0	27
18	0	0	0	0	29
19	0	0	0	0	16
20	0	0	0	0	0
21	0	0	0	0	0
22	0	0	0	0	0
23	0	0	0	0	0
24	0	0	0	0	0

Table 8: $\lambda(\Gamma^u)$ for $n = 5$

values, we have to omit the first parts of our bit row,¹⁶ depending on what recursion level m_i we are at. If we would not omit the first parts, then we would have to face interdependency between m_i and previous m_j , $j < i$, when we are currently at $v \in M \times W$, $m_i \in v$. Since Γ is of symmetrical nature, we can neglect the lower half of Γ without any loss of information.

We start with the first element $m_1 \in M$. The algorithm proceeds the first n rows (from $y = 0$ to $y = 4$) and determines all vertices $v \in V$ belonging to the numerical value $\lambda_2(\Gamma_y)$ for each $\{m_1, w_y(m_1)\}$.¹⁷

Each new row y leads to a new recursion at the second recursion depth. As an illustration in our example, we show the step of processing the second row $y = 1$ where $v = \{m_1, w_2\}$ after our algorithm successfully processed the first row $y' = 0$ where $v' = \{m_1, w_1\}$.¹⁸ In table 9, we clearly see under the second column of the bit values belonging to m_2 in row

¹⁶That is, technically speaking, $\lambda_x(\Gamma_y) = 0, \forall x < \lfloor y/n \rfloor$.

¹⁷The assigned w to $m_i \in M$, $i \in [1, n]$ in row y of Γ (or $\lambda(\Gamma)$) is denoted by $w_y(m_i)$.

¹⁸Due to a better handling with bit operations and the correct calculation of the next row to be processed, the row index y starts with 0 instead of 1 as it might be usual in mathematical notations.

$y = 1$, i.e. $b_2(m_1, w_2) = [10111]$, that our next vertices $v \in V, m_2 \in v$, to be processed comprise $w_1, w_3, w_4, w_5 \in W$ since they have a stable linkage to $\{m_1, w_2\}$. Before we go to the next recursion step at m_3 , we have to examine which vertices that will be processed in the upcoming steps are shared by the current vertices in chain c . This can be easily done by the intersection of the bit rows of all elements belonging to chain c . In the current step, this is $\{m_1, w_2\}$ and the obtained vertices from rows 5 ($\{m_2, w_1\}$), 7 ($\{m_2, w_3\}$) and 8 ($\{m_2, w_4\}$) for each.¹⁹ So we have a loop that runs over the vertices $\{m_2, w_1\}$, $\{m_2, w_3\}$ and $\{m_2, w_4\}$ and, for each vertex $v, m_2 \in v$, a new bit mask $b'(v) = b(m_1, w_2) \wedge b(v)$ is created that is given as a parameter to the function call for the next recursion step in m_3 .

From the third column under the bit vector of $\Gamma_5^{u'}$, $\Gamma_7^{u'}$ and $\Gamma_8^{u'}$, we can see that there is only one single vertex $v = \{m_3, w_5\}$ that might be part in a stable match $\mu \in \mathcal{M}, \forall v, m_2 \in v$, since we have the same bit value $b'_3(v) = [00001], \forall v \in V, m_2 \in v$. For the next recursion layer, the next bit mask $b''(m_3, w_5) = b'(v) \wedge \Gamma_{14}$ has to be computed that in turn is given as a parameter to the function call for the recursion in m_4 and so on.

This algorithm successively outputs a stable match when $|c| = n$, or it continues to the next v' in the current layer when $\lambda_x(b'(v)) = 0$, for any $x \in [1, n]$ and $v \in M \times W$, after some bitwise intersection operations. After the completion of the traversal of the matching tree, the algorithm finishes its procedure.

In table 10, we see the single operations after every recursion step. For a better understanding, we do not list all variables from the loop but we pick only one partial result and show the next procedural step to be conducted. In figure 2 that is derived from figure 1,²⁰ we show the colored bits that are relevant for the next recursion level and, for the same time, the bits that are significant for later synchronizations.

5 Additional remarks

On <https://github.com/martinwerner1/matching>, further papers and test results are presented under GPL-2.0 license to the public.

¹⁹By lemma 3, vertex $v = \{m_2, w_5\}$ is removed due to $\lambda_3(\Gamma_9) = 0$.

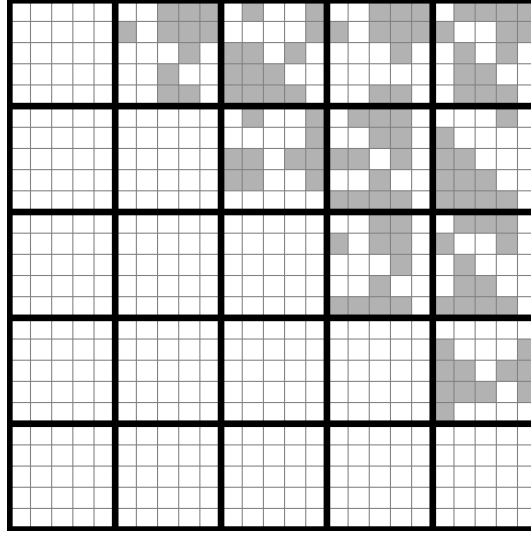
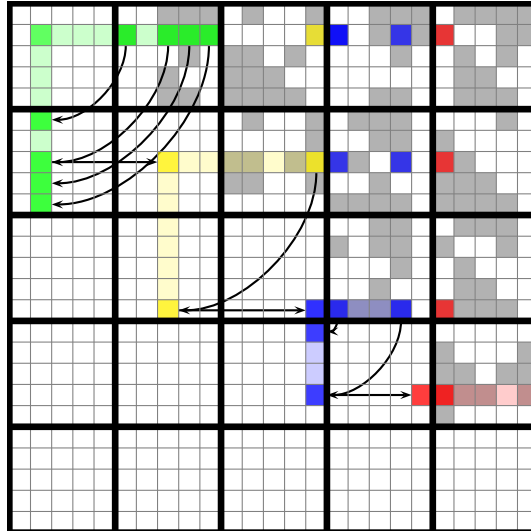
²⁰Note that the grey fields in the subgrid are a representation for bits that are valued 1.

		Bit values					Numerical values					
Γ_y^u	$b(v), v \in M \times W$	m_1	m_2	m_3	m_4	m_5	$\lambda(\Gamma_y^u)$	m_1	m_2	m_3	m_4	m_5
Γ_1^u	$b(m_1, w_2)$	[00000]	[10111]	[00001]	[10111]	[10011]	$\lambda(\Gamma_1^u)$	0	23	1	23	19
Γ_5^u	$b(m_2, w_1)$	[00000]	[00000]	[01001]	[01110]	[00010]	$\lambda(\Gamma_5^u)$	0	0	9	14	2
Γ_7^u	$b(m_2, w_3)$	[00000]	[00000]	[11011]	[11010]	[11000]	$\lambda(\Gamma_7^u)$	0	0	27	26	24
Γ_8^u	$b(m_2, w_4)$	[00000]	[00000]	[11001]	[00100]	[11100]	$\lambda(\Gamma_8^u)$	0	0	25	4	28
$\Gamma_5^{u'}$	$b(m_1, w_2) \wedge b(m_2, w_1)$	[00000]	[00000]	[00001]	[00110]	[00010]	$\lambda(\Gamma_5^{u'})$	0	0	1	6	2
$\Gamma_7^{u'}$	$b(m_1, w_2) \wedge b(m_2, w_3)$	[00000]	[00000]	[00001]	[10010]	[10000]	$\lambda(\Gamma_7^{u'})$	0	0	1	18	16
$\Gamma_8^{u'}$	$b(m_1, w_2) \wedge b(m_2, w_4)$	[00000]	[00000]	[00001]	[00100]	[10000]	$\lambda(\Gamma_8^{u'})$	0	0	1	4	16

Table 9: Bit operations on certain rows y of Γ^u as well as $\lambda(\Gamma^u)$ **Algorithm 1:** BitSync algorithm (with additional constraints)**Function** *BitSync***Input:** Γ, b, c, y, n **Output:** $\mathcal{M}$ Initialize $\mathcal{M} = \emptyset$;**if** $|c| = n$ **then** Add c to $\mathcal{M}$; Return $\mathcal{M}$;**else** Initialize bit vector $b' \leftarrow \emptyset$; Add the intersection to new bit mask b' s.t. $b' \leftarrow \Gamma[y] \wedge b$; Initialize $m \leftarrow \lfloor y/n \rfloor$; **if** $\lambda_x(\Gamma_y) \neq 0, \forall x \in [m+1, n]$ **then** **for** For every bit position i in mask $\lambda_{m+1}(b'), b'_{m+1}[i] = 1$ **do** Determine new row y' concerning vertices v belonging to $\Gamma[y][b'_{m+1}[i]]$; $c' \leftarrow c \cup v$; Add *BitSync*(Γ, b', c', y', n) to $\mathcal{M}$; Return $\mathcal{M}$;**End Function**

		Bit values					Numerical values					
Γ_y^u	$b(v), v \in M \times W$	m_1	m_2	m_3	m_4	m_5	$\lambda(\Gamma_y^u)$	m_1	m_2	m_3	m_4	m_5
Γ_1^u	$b(m_1, w_2)$	[00000]	[10111]	[00001]	[10111]	[10011]	$\lambda(\Gamma_1^u)$	0	23	1	23	19
By $b_2(\Gamma_1^u) = [10111]$, we have w_1, w_3, w_4, w_5 for m_2 and $c = \{\Gamma_1\}$, we pick one and proceed with $\{m_2, w_3\}$.												
Γ_7^u	$b(m_2, w_3)$	[00000]	[00000]	[11011]	[11010]	[11000]	$\lambda(\Gamma_7^u)$	0	0	27	26	24
$\Gamma_7^{u'}$	$b(m_1, w_2) \wedge b(m_2, w_3)$	[00000]	[00000]	[00001]	[10010]	[10000]	$\lambda(\Gamma_7^{u'})$	0	0	1	18	16
By $b_3(\Gamma_7^{u'}) = [00001]$, we have w_5 for m_3 and $c = \{\Gamma_1, \Gamma_7\}$, we proceed with $\{m_3, w_5\}$.												
Γ_{14}^u	$b(m_3, w_5)$	[00000]	[00000]	[00000]	[11110]	[11110]	$\lambda(\Gamma_{14}^u)$	0	0	0	30	30
$\Gamma_{14}^{u''}$	$b'(m_2, w_3) \wedge b(m_3, w_5)$	[00000]	[00000]	[00000]	[10010]	[10000]	$\lambda(\Gamma_{14}^{u''})$	0	0	0	18	16
By $b_4(\Gamma_{14}^{u''}) = [10010]$, we have w_1, w_4 for m_4 and $c = \{\Gamma_1, \Gamma_7, \Gamma_{14}\}$, we pick one and proceed with $\{m_4, w_4\}$.												
Γ_{18}^u	$b(m_4, w_4)$	[00000]	[00000]	[00000]	[00000]	[11101]	$\lambda(\Gamma_{18}^u)$	0	0	0	0	29
$\Gamma_{18}^{u'''}$	$b''(m_3, w_5) \wedge b(m_4, w_4)$	[00000]	[00000]	[00000]	[00000]	[10000]	$\lambda(\Gamma_{18}^{u'''})$	0	0	0	0	16
By $b_5(\Gamma_{18}^{u'''}) = [10000]$, we have w_1 for m_5 and $c = \{\Gamma_1, \Gamma_7, \Gamma_{14}, \Gamma_{18}\}$, we proceed with $\{m_5, w_1\}$.												
We have $c = \{\Gamma_1, \Gamma_7, \Gamma_{14}, \Gamma_{18}, \Gamma_{20}\}$ and $ c = n = 5$. Stable match!												

Table 10: Bitwise intersection operations until match μ

Figure 1: Stability matrix Γ^u for the given example in table 3Figure 2: Stability matrix Γ^u during BitSync operation